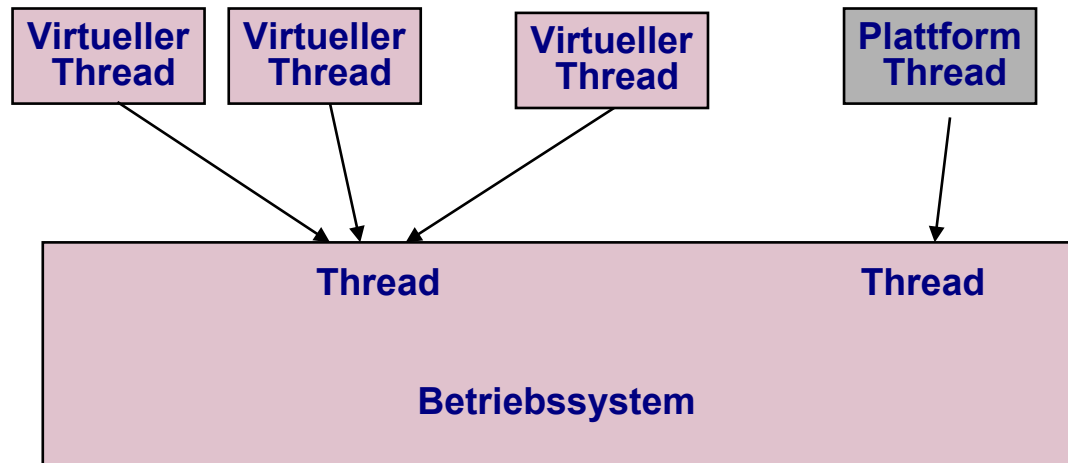




Java – Neues in den Bibliotheken (21 - 25)

Stephan Karrer

Virtuelle Threads (ab Java 21)



Eigenschaften:

- Bisheriges Thread-Konzept bildet Java-Threads (Platform Threads) meist 1:1 auf BS-Threads ab (teure Ressource)
- Ein virtueller Thread wird nur während seiner Aktivität ein BS-Thread zugeordnet, im blockierten Zustand zwecks I/O nicht (vergleichbar mit virtuellem Speicher)
- Virtuelle Threads sind nicht schneller als Plattform-Threads, können aber gerade bei I/O-lastigen Code den Durchsatz erhöhen
- Virtuelle Threads sind auch Instanzen der Klasse `java.lang.Thread` und wir haben syntaktisch das gleiche Programmiermodell

Thread erzeugen via Thread.Builder (ab Java 21)

```
Thread.Builder builder =  
    Thread.ofVirtual().name("MyThread");  
Runnable task = () -> {  
    System.out.println("running vt1"); };  
Thread virtualThread1 = builder.start(task);  
  
// oder kompakter  
Thread virtualThread2 =  
    Thread.ofVirtual().start(  
        () -> System.out.println("running vt2")); ;  
  
// funktioniert auch für platform thread  
Thread platformThread =  
    Thread.ofPlatform().start(  
        () -> System.out.println("running pt")); ;}
```

- Virtuelle Threads bilden eine Threadgruppe, haben stets **NORM_PRIORITY** und sind Dämonen
- Virtuelle Threads können nicht per Konstruktoren der Klasse **Thread** erzeugt werden, sondern nur per statischen Methoden
- Via **Thread.Builder** können sowohl Plattform als auch virtuelle Threads erzeugt werden
- Es gibt auch die Variante mit reiner Erzeugung ohne Start
- **Thread.Builder** ist ein Beispiel für ein Sealed Interface und in der Klasse **Thread** enthalten

Threadaufbau via Builder

```
Thread plbuild = Thread.ofPlatform()  
    .name("plBuild")  
    .group(Thread.currentThread().getThreadGroup())  
    .inheritInheritableThreadLocals(false)  
    .priority(Thread.MAX_PRIORITY)  
    .stackSize(0)  
    .uncaughtExceptionHandler(  
        (t, ex) -> System.err.println(ex))  
    .unstarted(  
        () -> System.out.println("running plbuild"));  
plbuild.start();
```

- Via Builder-Pattern kann der Thread mit seinen Properties schrittweise aufgebaut werden

Virtuelle Threads erzeugen via Executor

```
ExecutorService service =  
    Executors.newVirtualThreadPerTaskExecutor();  
  
for (int i = 0; i < 100; i++) {  
    service.submit(() -> {  
        long id = Thread.currentThread().threadId();  
        LockSupport.parkNanos(1_000_000_000); //simuliert Blocking  
        System.out.println(id);  
    });  
}  
  
service.close();
```

- Es existiert ein eigener **ExecutorService** für die Erzeugung eines Thread-Pools
- Bei Bedarf auch eigener Service via **ThreadFactory** aufgebaut werden (bereits seit Java 1.5 verfügbar)

Virtuelle Threads: Pinning

- Der Scheduler der VM bindet (mount) virtuelle Threads an Träger-Threads, deren Anzahl entspricht der Anzahl CPU-Cores per Default. Wenn der virtuelle Thread blockiert, wird üblicherweise der Träger-Thread frei und kann andere virtuelle Threads bedienen.
- Das ist aber nicht immer möglich und der virtuelle Thread ist „pinned“ und der Träger-Thread ist blockiert:
 - Bei Ausführung eines synchronized Blocks bzw. Methode (Java 21 – 23)
 - Bei Ausführung einer nativen Methode (oder einer foreign function, Java 22)
- Bis Java 23 statt synchronized besser Reentrant Locks benutzen

siehe auch: <https://dev.java/learn/new-features/virtual-threads/>

Goldene Regeln zur Verwendung virtueller Threads

- Nutze virtuelle Threads zur Durchsatz-Steigerung bei vielen Tasks mit I/O-Blockierung bzw. Netzwerkzugriff
- Bei CPU-intensiven Tasks ist keine Verbesserung zu erwarten
- Der Vorteil: vertrautes synchrones Programmiermodell ohne Callbacks
- Nutze keine Threadpools für virtuelle Threads, für potentielle Zugriffs-Limitierung (rate limiting) können andere Mechanismen, z.B. Semaphore, genutzt werden
- Vorsicht mit potentielltem Overhead bei thread-lokalen Variablen, besser Scoped Variablen benutzen
- Analyse mit JDK Flight Recorder/JDK Mission Control oder anderen modernen Profilern, die virtuelle Threads kennen (rudimentär mit jcmd-Kommando)

Scoped Values (ab Java 25)

```
static final ScopedValue<String> REQ_ID = ScopedValue.newInstance();

public static void main(String[] args) {
    ScopedValue.where(REQ_ID, "4711").run(() -> f1()); }

private static void f1() { f2(); } //nutzt Value nicht

private static void f2() { System.out.println(REQ_ID.get()); }
```

Problem: „Unsichtbare“ Parameter über mehrere Methodenaufrufe durchreichen

Bisheriger Ansatz: Thread Local Variable

- Pro Thread eigene Kopie
- Grundsätzlich les- und schreibbar
- Ohne expliziten Remove existieren sie solange der Thread existiert
- Problematisch bei vielen virtuellen Threads

Neuer Ansatz: Scoped Value

- Pro Thread eigene Kopie der Referenz
- Nur lesbar (immutable)
- Existieren nur während der Ausführung der run-Methode (scoped)
- Berücksicht somit speziell virtuelle Threads

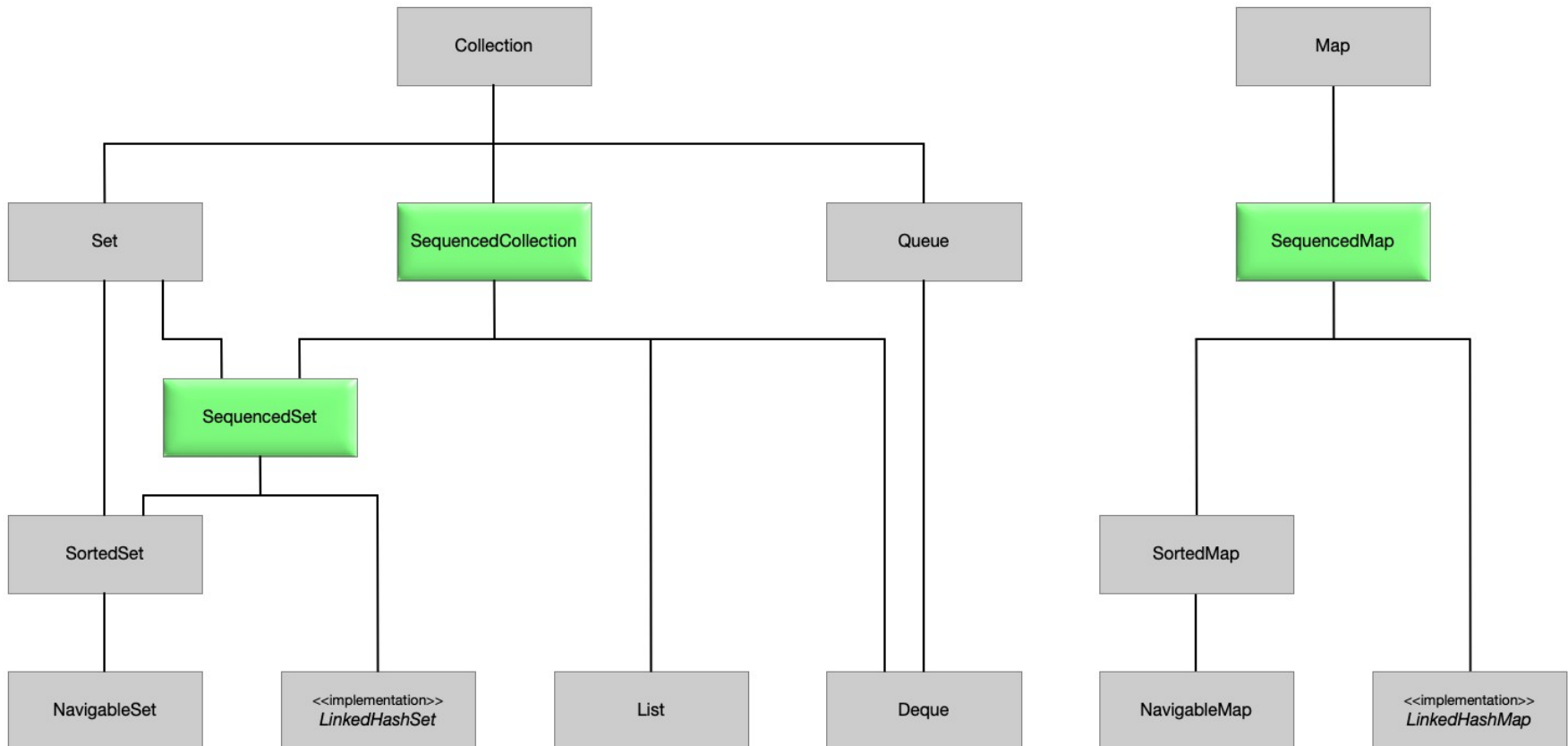
Sequenced Collections (ab Java 21)

- Ein neues Basis-Interface bei den Collections
- Es gibt eine Reihenfolge innerhalb der Elemente und der Zugriff bzw. umgekehrte Iteration wird vereinheitlicht
- **reversed()** liefert einen View über der eigentlichen Collection zur inversen Iteration
- Es gibt wiederum entsprechende Kind-Interfaces für Maps und Sets
- Wie bisher schon führt das zu unsauberen Design-Entscheidungen und jetzt zusätzlich zu möglicher Inkompabilität mit vorhandenem Code

```
interface SequencedCollection<E> extends  
Collection<E> {  
    // new method  
    SequencedCollection<E> reversed();  
    // methods promoted from Deque  
    void addFirst(E);  
    void addLast(E);  
    E getFirst();  
    E getLast();  
    E removeFirst();  
    E removeLast();  
}
```

```
//Beispiel: Vereinfachung für LinkedList  
linkedHashSet.reversed().stream();
```

Wie passt das in die vorhandene Interface-Hierarchie



Sonstige API-Erweiterungen

- Java 24 : Stream Gatherers (JEP 485)
 - Erweiterung der Stream-API um eigene Intermediate Operationen
- Java 24: Class-File API (JEP 484)
 - API zum Parsen, Generieren und Transformieren von Java class-Dateien
- Java 22: Foreign Function & Memory API
 - API mit dessen Hilfe Java-Programme mit Code und Speicher ausserhalb der VM operieren können. Dadurch können Entwickler ohne JNI native Bibliotheken und nativen Speicher zugreifen